

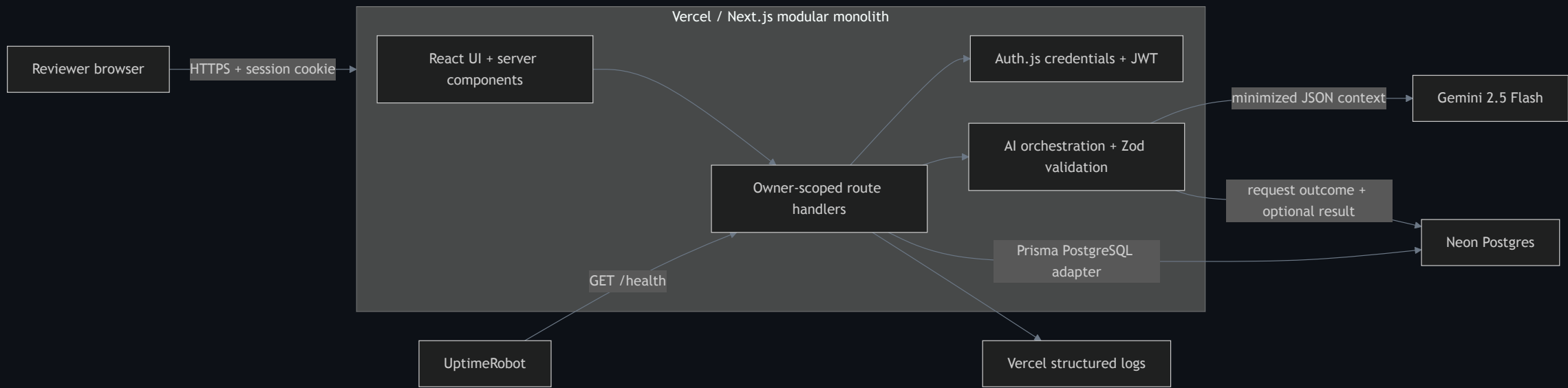
SYSTEM DESIGN

How it's built

One deployment, owner-scoped data, and a Gemini layer built to fail safely.

ARCHITECTURE

One Next.js app, one database



AUTH

Signing in

src/lib/auth.ts

```
22     async authorize(raw) {
23         const parsed = credentialsSchema.safeParse(raw);
24         if (!parsed.success) return null;
25         const user = await db.user.findUnique({ where: { email: parsed.data.email.toLowerCase() } });
26         if (!user || !(await compare(parsed.data.password, user.passwordHash))) return null;
27         return { id: user.id, email: user.email, name: user.name };
28     },
```

Two tables just for AI

```
prisma/schema.prisma

88  model AIResult {
89    id          String      @id @default(cuid())
90    ownerId     String
91    owner       User        @relation(fields: [ownerId], references: [id], onDelete: Cascade)
92    leadId      String?
93    lead        Lead?       @relation(fields: [leadId], references: [id], onDelete: Cascade)
94    useCase     AiUseCase
95    model       String
96    schemaVersion Int       @default(1)
97    result      Json
98    createdAt   DateTime    @default(now())
99
100   @@index([ownerId, useCase, createdAt])
101   @@index([leadId, useCase, createdAt])
102 }
103
104  model AIRequest {
105    id          String      @id @default(cuid())
106    ownerId     String
107    owner       User        @relation(fields: [ownerId], references: [id], onDelete: Cascade)
108    useCase     AiUseCase
109    model       String?
110    resultCategory String
111    durationMs   Int?
112    retryCount   Int        @default(0)
113    createdAt   DateTime    @default(now())
114
115   @@index([ownerId, createdAt])
116 }
```

AUTHORIZATION

Owner scoping and stale writes

src/app/api/leads/[id]/route.ts

```
export async function PATCH(request: Request, { params }: Context) {
  try {
    const ownerId = await requireUserId();
    const { id } = await params;
    const input = leadInputSchema.parse(await request.json());
    const result = await db.lead.updateMany({ where: { id, ownerId, archivedAt: null, ...(input.updatedAt ? { updatedAt: input.updatedAt } : {}) }, data: { name: input.name, company: input.company, email: cleanOptional(input.email), phone: cleanOptional(input.phone), city: cleanOptional(input.city), industry: cleanOptional(input.industry), source: cleanOptional(input.source), valuePaise: input.valuePaise, status: input.status, notes: cleanOptional(input.notes) } });
    if (!result.count) {
      const exists = await db.lead.count({ where: { id, ownerId, archivedAt: null } });
      throw new Error(exists ? "EDIT_CONFLICT" : "NOT_FOUND");
    }
    return Response.json({ data: await db.lead.findUnique({ where: { id } }) });
  } catch (error) { return apiError(error); }
}
```

PRIVACY

What Gemini actually receives

src/lib/ai/context.ts

```
function sanitizeText(value: string | null | undefined, max: number) {
  return (value ?? "").replace(/[\u0000-\u001F\u007F]/g, " ").replace(/\s+/g, " ").trim().slice(0, max);
}

export function buildLeadContext(lead: LeadForAi) {
  return {
    leadId: lead.id,
    company: sanitizeText(lead.company, 120),
    city: sanitizeText(lead.city, 80),
    industry: sanitizeText(lead.industry, 80),
    source: sanitizeText(lead.source, 80),
    status: lead.status,
    valueBand: lead.valuePaise == null ? "UNKNOWN" : lead.valuePaise < 5_000_000 ? "UNDER_50K_INR" : lead.valuePaise < 25_000_000 ?
"50K_TO_250K_INR" : "OVER_250K_INR",
    notes: sanitizeText(lead.notes, 600),
    recentFollowUps: (lead.followUps ?? []).slice(0, 6).map((item) => ({
      kind: item.kind,
      note: sanitizeText(item.note, 180),
      dueAt: item.dueAt.toISOString(),
      completed: Boolean(item.completedAt),
    })),
  };
}
```

STRUCTURED OUTPUT

Every response has a contract

src/lib/ai/schemas.ts

```
3 export const leadInsightSchema = z.object({
4   opportunity: z.string().trim().min(1).max(240),
5   risk: z.string().trim().min(1).max(240),
6   evidence: z.array(z.string().trim().min(1).max(180)).min(1).max(5),
7   recommendedNextAction: z.string().trim().min(1).max(240),
8   confidence: z.enum(["LOW", "MEDIUM", "HIGH"]),
9   caveat: z.string().trim().min(1).max(240),
10  });
```

RESILIENCE

Checks before the call is made

src/lib/ai/gemini.ts

```
25     const recentCount = await db.aiRequest.count({ where: { ownerId, createdAt: { gte: new Date(Date.now() - 60_000) } } });
26     if (isAiRateLimited(recentCount)) throw new Error("AI_RATE_LIMIT");
27     if (!process.env.GEMINI_API_KEY) throw new AiGenerationError("UNAVAILABLE", "GEMINI_NOT_CONFIGURED");
28
29     const request = await db.aiRequest.create({ data: { ownerId, useCase, model: MODEL, resultCategory: "STARTED" } });
30     const started = Date.now();
```


RESILIENCE

Only retry what's worth retrying

src/lib/ai/gemini.ts

```
46     } catch (error) {
47       const message = error instanceof Error ? error.message : "";
48       const category: AiErrorCategory = message.startsWith("INVALID_RESPONSE") ? "INVALID_RESPONSE" : classifyAiError(error);
49       if (attempt === 0 && (category === "TRANSIENT" || category === "RATE_LIMITED")) {
50         retryCount = 1;
51         await wait(300 + Math.floor(Math.random() * 350));
52         continue;
53       }
54       throw new AiGenerationError(category);
```

FALLBACKS

When Gemini is unavailable

src/lib/ai/fallbacks.ts

```
export function leadInsightFallback(lead: Lead): LeadInsight {
  const next = lead.followUps?.find((item) => !item.completedAt);
  return { opportunity: `${lead.company} is currently in the ${lead.status.toLowerCase()} stage.`, risk: next ? "The next commitment needs timely follow-through." : "No open follow-up is scheduled.", evidence: [`Pipeline stage: ${lead.status}`, next ? `Open follow-up: ${next.note}` : "No open follow-up"], recommendedNextAction: next ? `Complete the next follow-up by ${next.dueAt.toLocaleDateString("en-IN")}` : "Schedule a specific next step with a due date.", confidence: "LOW", caveat: "Gemini was unavailable, so this is a rules-based fallback using CRM fields only." };
}
```

FALLBACKS

And a test that holds them to it

tests/ai-fallbacks.test.ts

```
8     it("returns a schema-valid lead insight grounded in CRM fields", () => {
9         const result = leadInsightFallback(lead);
10        expect(leadInsightSchema.parse(result)).toEqual(result);
11        expect(result.evidence.join(" ")).toContain("PROPOSAL");
12        expect(result.caveat).toContain("rules-based");
13    });
```

DEPLOY AND MONITORING

Shipping it and watching it

- **One Vercel deploy** plus one Neon Postgres. Both free tier, both inside the brief's constraints.
- **Migrations are deployable.** Prisma migrate on the database, seed is repeatable and only touches the demo user.
- **Public health endpoint.** `/api/health` pings the database and reports app version and latency.
- **GitHub Actions hits it every 5 minutes**, retries transient failures, and fails the run unless status and database both read ok. Run history is public.
- **Structured JSON logs** to Vercel, plus every AI call's outcome and duration queryable in Postgres.

QUALITY

Tested where it counts, and honest where it isn't

95.4%

branch coverage on domain and AI logic

6.34%

global line coverage, the real gap

99

Lighthouse performance on /login

- Domain rules and AI contracts are unit tested. That's where a silent bug costs most.
- Routes, pages and components have **no** unit tests. They were checked by hand.
- Both numbers are written down in DECISIONS.md rather than rounded up.

NEXT STEPS

What I'd change for real scale

- **Tenants.** A tenant model plus Postgres row-level security, so isolation isn't just discipline.
- **Queue the AI.** Move generation off the request path and stream status back.
- **Redis limits.** The DB counter is correct but not free at volume.
- **Real observability.** Traces, cost metrics, and prompt regression evals.
- **Not microservices.** Only split when something measured says to.

THANKS FOR WATCHING

Vaada

Every promise, followed through.